

BrisConnect Firestore to MySQL Migration Notes

1) Migration Goal

Move the current Firestore-backed data model to MySQL while preserving:

- Existing app behavior and permissions.
- Existing IDs and references where possible.
- Auditability (created/updated timestamps, moderation actions).
- Data quality for analytics and reporting.

2) Recommended Strategy (Phased)

1. Phase A: Prepare schema and staging

- Deploy `docs/BrisConnect_MySQL_Schema.sql` to a staging MySQL instance.
- Add a one-time `source_firestore_path` column in staging tables if you want row-level traceability during migration.
- Create read-only DB user for verification queries.

2. Phase B: Backfill historical data

- Export Firestore collections to JSON/CSV snapshots.
- Run ETL scripts collection-by-collection in dependency order.
- Re-run ETL idempotently until counts and checksums match.

3. Phase C: Dual-write window

- Update backend service layer to write to Firestore and MySQL.
- Keep reads on Firestore first.
- Monitor divergence using nightly reconciliation jobs.

4. Phase D: Read cutover

- Switch low-risk reads first (reference lists, public content).
- Switch transactional reads next (events, reports, notifications).
- Keep Firestore writes for a short fallback window.

5. Phase E: Final cutover and decommission

- Stop Firestore writes.
- Keep Firestore snapshots archived for compliance/rollback.
- Remove dual-write once stability KPIs are met.

3) Entity Migration Order

Use this order to satisfy foreign keys and avoid temporary null references:

1. `admins`
2. `local_users` , `visitor_users`
3. `attractions`
4. `events`
5. `attraction_details` , `discover_items`
6. `event_reports` , `app_feedback` , `user_notifications`
7. Junction tables:
 - `local_user_interested_events`
 - `visitor_user_interested_events`

- `visitor_user_saved_attractions`
- `local_user_interest_categories`
- `visitor_user_interest_categories`

8. Supporting tables:

- `seed_metadata` , `counters` , `app_config`
- `mail_queue` , `sms_queue`
- `brisbane_stories` , `brisbane_voices` , `connectivity_probe`

4) Firestore to MySQL Mapping Rules

IDs and keys

- Firestore document IDs should be retained as primary keys in MySQL (`VARCHAR`).
- Collection/document paths should not be used as keys in final schema (only optional trace columns during migration).

Timestamps

- Firestore `Timestamp` should map to `DATETIME(3)` .
- Preserve UTC on both write and read paths.

Arrays

- Arrays used for relationships should become junction tables.
- Arrays of primitive metadata can remain in `JSON` if not query-critical.

Maps/objects

- Flexible nested objects should map to `JSON` columns (for example details payloads).
- Frequently filtered fields should be promoted to first-class columns.

Booleans/enums

- Firestore boolean fields map to `BOOLEAN` .
- Status strings should be normalized to controlled enum-like values in app validation.

5) ETL Implementation Pattern

Use a repeatable ETL pattern per collection:

1. Extract
 - Pull full collection snapshot with document ID + payload.
2. Transform
 - Apply type normalization.
 - Split arrays into relational rows for junction tables.
 - Coerce missing optional fields to `NULL` .
3. Load
 - Use `INSERT ... ON DUPLICATE KEY UPDATE` for idempotency.
 - Batch loads in consistent chunk sizes (for example 500 to 2000 rows).
4. Validate

- Row counts per collection/table.
- Field-level spot checks on critical entities.
- Referential integrity checks (orphan detection queries).

6) Index and Query Considerations

- Recreate Firestore composite query behavior with MySQL indexes.
- Existing known composite from Firestore:
 - `event_reports(status ASC, createdAt DESC)`
- Equivalent MySQL index recommendation:
 - `CREATE INDEX idx_event_reports_status_created_at ON event_reports(status, created_at DESC);`
- Add indexes gradually based on observed API query plans (`EXPLAIN`).

7) Data Quality and Reconciliation

Run these checks during dual-write:

1. Count parity
 - Compare Firestore document counts vs MySQL row counts by domain.
2. Referential parity
 - Verify every `event_reports.event_id` exists in `events.id` .
 - Verify every notification `event_id` (when not null) exists in `events.id` .
3. Business parity
 - Compare approved/rejected/open counts for moderation entities.
 - Compare active upcoming events by date range.
4. Sample payload parity
 - Randomly sample records and compare key fields and JSON payloads.

8) Security and Access Model Changes

- Firestore Rules are replaced by application-layer authorization + DB grants.
- Enforce role checks in service code (admin/local/visitor).
- Use least-privilege DB users:
 - app runtime user (CRUD on app tables)
 - migration user (bulk write during ETL)
 - read-only analytics user

9) Rollback Plan

If cutover issues occur:

1. Switch reads back to Firestore using feature flag.
2. Continue dual-write while fixes are applied.
3. Rebuild inconsistent tables from latest Firestore snapshot.
4. Repeat reconciliation before retrying cutover.

10) Cutover Readiness Checklist

- ☐ Schema applied successfully in staging and production.

- ☐ Backfill completed with parity checks passed.
- ☐ Dual-write enabled and stable for agreed soak period.
- ☐ No unresolved orphan/reference errors.
- ☐ API performance meets baseline SLO after read switch.
- ☐ Rollback switch tested and documented.